Data Structures

Lecture 9

Searching

Abstract Data Type

References:
Data Structures and Algorithms in Java
Michael T. Goodrich and Roberto Tamassia.

University of Kufa
Faculty of Computer Science and Mathematics

Dr. Munqath Al-Attar
Information Technology Research and Development Center ITRDC
Email:
munqith.alattar@uokufa.edu.iq
Website:
http://staff.uokufa.edu.iq/profile.html?munqith.alattar

Topics

- Postfix, Prefix and Infix Notations
- Linked List Representation of a Binary Tree

- Searching:
  - Sequential search
  - Binary Search
- Exercises

Data Structures
Postfix, Prefix and Infix Notations

- **Infix notation:** the operator comes between the two operands, example: a+b

  Needs to consider Operators' Priority and Associative property

  - Expression 2+3*4 evaluate to:

    (2+3)*4 = 20

    2+(3*4) = 14

- **Postfix notation:** the operator comes after its two operands example: ab+

- **Prefix notation:** the operator coms before its two operands, example: +ab

Data Structures
Postfix, Prefix and Infix Notations

Infix Expression	Prefix Expression	Postfix Expression
A + B * C + D	+ + A * B C D	A B C * + D +
(A + B) * (C + D)	* + A B + C D	A B + C D + *
A * B + C * D	+ * A B * C D	A B * C D * +
A + B + C + D	+ + + A B C D	A B + C + D +

Infix to postfix conversation

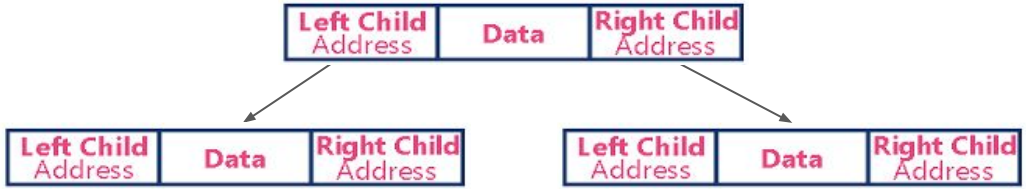
Example: Input infix expression : a * (b + c + d) Output postfix expression : abc+d+*

Assignment: Write the algorithm that convert an Infix expression to Postfix Expression with an example. (Use Stack)

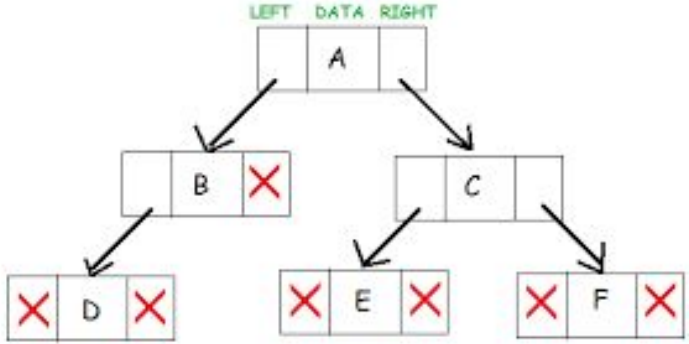
Data Structures
Linked List Representation of a Binary Tree

It is common to represent a binary tree using linked list, where every element is represented as nodes that contains :

- **Left Child (Address):** points to the address of the left child of the parent node
- **Information of the Node:** holds the information of the node
- **Right Child (Address):** points to the address of the right child of the parent node



If a node has no left or right child, then the corresponding left or right address is NULL.

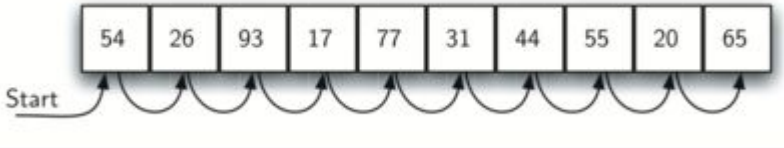


Data Structures
Searching

The process of finding the desired information from the set of items stored in the form of elements (such as: array, tree, graph, or linked list) in the computer memory.

Searching algorithms are generally classified into two categories:

1. Sequential Search (For example: Linear Search)
2. Interval Search (For example: Binary Search)



1. **Sequential search (Linear Search)**

- The most simple search algorithm in data structure
- The list of elements is traversed sequentially to check every element of the set.
- TSpace complexity for linear search is O(n).

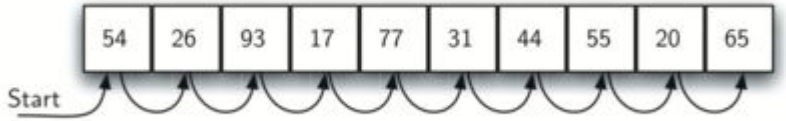
Linear Search Algorithm

```
procedure linear_search (list, value)

  for each item in the list
    if match item == value
      return the item's location
    end if
  end for

end procedure
```

Example, To find '31' the linear search algorithm will check each element sequentially till its pointer points to 31.



Interval Search (Binary Search)

repeatedly target the center of the search structure and divide the search space in half.

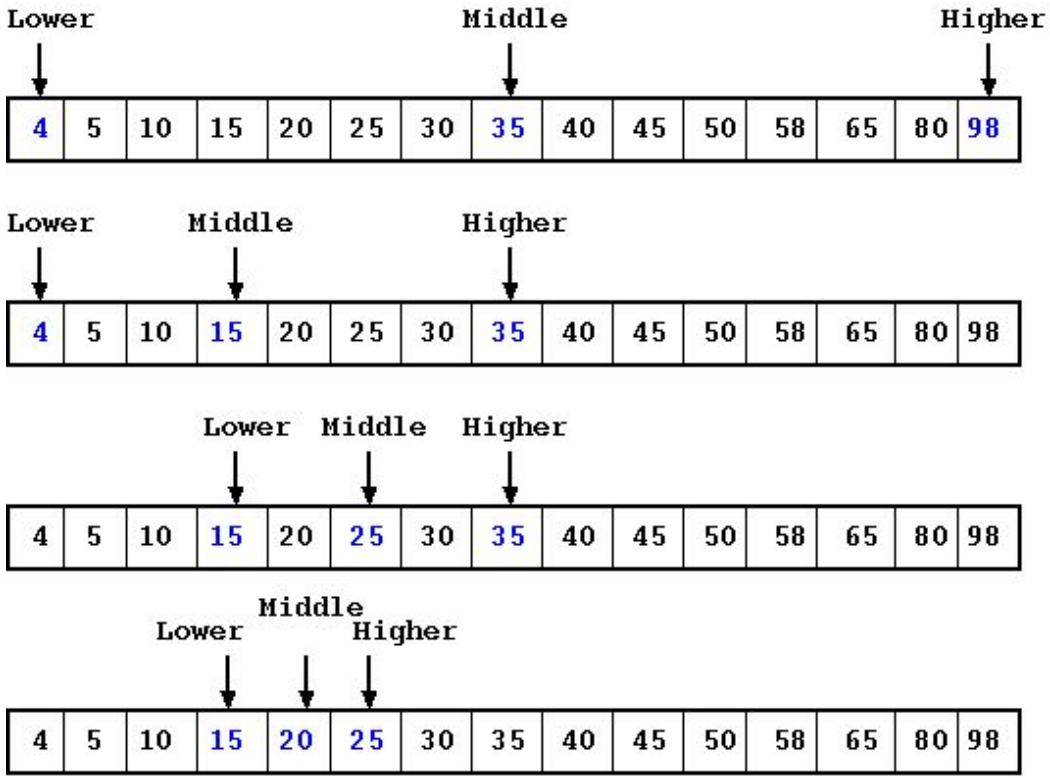
Binary Search Algorithm

- Begin the search with the mid element of the array.
- Repeat until the value is found or the interval is empty:
  - If the searching value equals to the item:
    - return an index of item.
  - ElseIf the searching value is less than the item in the middle of the interval:
    - narrow the interval to the lower half
  - Else:
    - narrow it to the upper half.

Data Structures
Searching

Example: Find the element “20” using Binary Search Algorithm

Exercise: Find the element “58” using Binary Search Algorithm



Exercises

1. **Insert items with the following keys (in the given order) into an initially empty binary search tree: 30 , 40 , 24 , 58 , 48 , 26 , 11 , 13 . Draw the tree after any two insertions.**

2. **Choose a set of 7 distinct, positive, integer keys. Draw binary search trees for your set of height 2 , 5 , and 6 .**